

COE/ELE 70AB

Milestones Compliance

Report 3

Victor Do (Project Manager)
Toronto Metropolitan University,
Software Engineering

Cameron Tuffner-Lyons
Toronto Metropolitan University,
Computer Engineering

Davis Cheung
Toronto Metropolitan University,
Computer Engineering

JJ Marr
Toronto Metropolitan University,
Software Engineering

Abstract—This paper introduces a heterogeneous CPU-GPU framework for collision detection in fluid simulations, designed to optimize computational efficiency by integrating broad- and narrow-phase collision detection across mixed device architectures. By partitioning the workload based on particle density, the model assigns low-density grid regions to the CPU, while high-density regions leverage the GPU's parallel processing capabilities for efficient particle interaction handling in dynamic scenes. Key architectural strategies include optimizing GPU SIMT (Single Instruction Multiple Threads) performance through memory layout techniques such as Structure-of-Arrays (SoA) formatting and cache alignment, enhancing overall processing speed. Initial benchmarks suggest that this approach significantly accelerates simulation performance, particularly in low-power environments. Future work will focus on refining the dynamic thresholding mechanism for more adaptive workload distribution and exploring further optimizations in collision resolution.

■ **INTRODUCTION**— Collision detection is a fundamental requirement for fluid simulation, especially in high-performance applications where precision and efficiency are essential. Traditional approaches often rely on either CPU or GPU resources independently, leading to limitations in scalability and speed. Recent developments have shown the potential of hybrid CPU-GPU architectures to overcome these challenges by distributing computational loads effectively. This

paper explores a heterogeneous approach to collision detection within fluid simulations, aiming to optimize performance in mixed device architectures. The paper is structured to guide the reader through the design and implementation of a heterogeneous CPU-GPU collision detection model for fluid simulations, beginning with an overview of collision detection algorithms and GPU architecture before concluding with our specific CPU-GPU integration methodology and preliminary findings.

Date of submission 01 Nov 2024; date of current version 01 Nov 2024

COLLISION DETECTION ALGORITHMS

A collision detection algorithm is a method used in computer graphics, physics simulations, and game development to determine whether two or more objects in a virtual environment are intersecting or coming into contact. This process is crucial for realistic interactions, ensuring that objects behave in a way that mimics real-world physics [1].

For the purpose of efficiency, collision detection is typically divided into two phases: broad phase and narrow phase [1].

After analyzing these algorithms will also find ways to divide the computational load between the CPU and GPU.

Broad Phase

The goal of this phase is to quickly eliminate pairs of objects that are far apart and cannot possibly collide. This phase uses simple geometric shapes or bounding volumes (like bounding boxes or spheres) to represent objects. By using these approximations, the algorithm can reduce the number of potential collisions that need to be checked in detail during the narrow phase. For example, if two bounding boxes do not overlap, there's no need to check for more detailed collisions between the actual objects [1].

Axis-Aligned Bounding Box: The AABB collision detection algorithm is used to detect overlap between two rectangular bounding-boxes by comparing their boundaries along each axis. This algorithm is computationally efficient for broad-phase collision detection, as it only requires simple min-max comparisons along each axis. However, it performs poorly for rotating or irregularly shaped objects because the boxes are axis-aligned. This can result in excessive empty space within the bounding box, leading to unnecessary collision checks and reduced accuracy [1].

Sweep and Prune: SAP is a technique that identifies potential collisions by projecting object bounds onto one or more coordinate axes. It sorts the objects along an axis, then "sweeps" through to identify overlapping intervals. Only objects with overlapping bounds on all axes are considered as potential collision candidates, reducing the number of pairs that require further narrow-phase testing. Sweep and Prune is highly efficient for scenes with limited movement, however it becomes less efficient in highly dynamic scenes where

objects move frequently, as the algorithm requires constant re-sorting, which can be computationally expensive. It is also less effective when objects are evenly distributed across space [1].

Bounding Volume Hierarchy: A BVH is a tree structure used in collision detection to manage complex scenes efficiently. Each node in the tree represents a bounding volume that contains a set of objects or other volumes, with the root node encompassing all scene objects. By subdividing space into nested volumes, BVH quickly eliminates non-colliding objects by first checking higher-level nodes, reducing collision checks in large scenes. BVH is effective for irregular or rotated objects, as it uses nested bounding volumes for better fit and precision. However, constructing and maintaining the BVH in dynamic scenes can be computationally intensive, and tree traversal often requires sequential checks, limiting full parallelism [1].

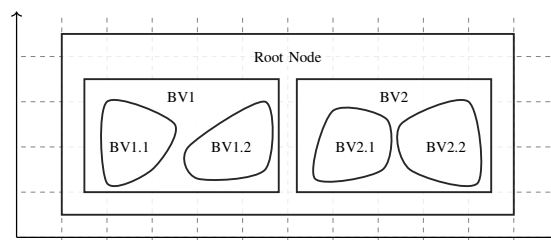


FIGURE 1. Bounding Volume Hierarchy

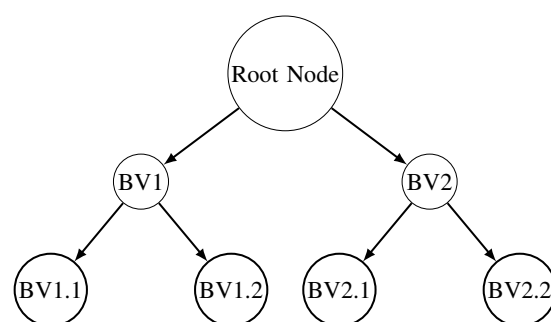


FIGURE 2. Tree Diagram for Bounded Volume Hierarchy

Quad-trees or Oct-trees: Quad-trees (2D) and oct-trees (3D) are spatial partitioning structures used to optimize collision detection by recursively dividing space into smaller regions. Each node represents a partition of space and subdivides further as more objects

are added, allowing collision checks to be limited to objects within the same or neighboring nodes. Quad-trees and oct-trees enhance collision detection efficiency by organizing objects spatially, enabling quick exclusion of distant objects. However, performance depends on balanced partitioning; poorly distributed objects can create uneven trees, with some nodes overloaded and others empty, reducing efficiency. They can also struggle in dynamic scenes where objects move frequently. This is because, with every significant movement, the tree may need to be restructured or rebalanced to maintain efficient spatial partitioning. Parallelizing quad-trees on the GPU is challenging due to their hierarchical and dynamic structure. Quad-trees rely on sequential dependencies, as nodes are recursively subdivided, which doesn't align well with the SIMD (Single Instruction, Multiple Data) architecture of GPUs [1].

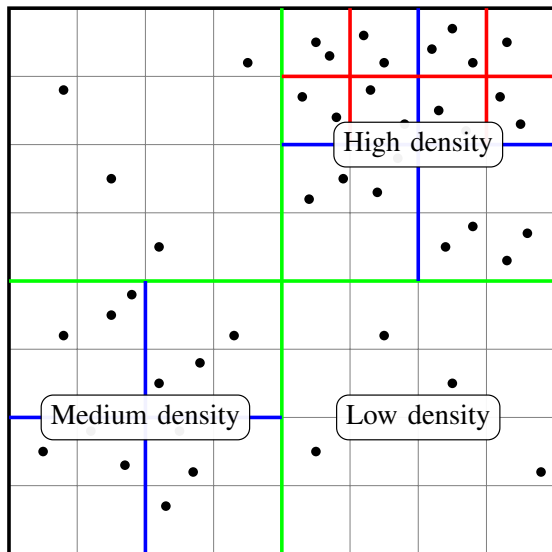


FIGURE 3. Quad-tree Subdivision showing areas with higher density being subdivided accordingly

Uniform Spatial Subdivision: Uniform spatial subdivision divides space into fixed-size grid cells, assigning objects to cells based on their positions. This approach simplifies spatial queries by restricting collision checks to objects within the same or neighboring cells. The fixed cell structure allows for efficient collision checks, limiting them to nearby objects, and is highly parallelizable, as collision checks for different cells are independent they can be processed concurrently on multi-core CPUs or GPUs. However, fixed-size

cells can be inefficient in unevenly distributed scenes: densely populated cells may suffer from performance bottlenecks, while empty cells waste resources. Additionally, cell size selection is crucial; large cells cause false positives, while small cells increase overhead. This rigidity limits adaptability to varying object densities and sizes [1].

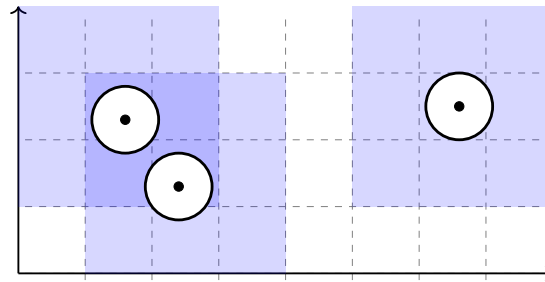


FIGURE 4. Uniform Spatial Subdivision grid with colliders and their adjacent cells highlighted

In summary,

spatial hashing or uniform grids are often preferred for GPU-based collision detection in dynamic scenes, as they allow for more straightforward parallel processing without the overhead of maintaining a tree structure. We overcome some of the inherent weaknesses of the technique as our application involves fluid particles which are uniform in size. For the fewer, variably-sized objects in the scene, we can use a secondary Sweep and Prune (SAP) algorithm to manage collisions effectively.

Narrow Phase

The narrow phase focuses on more accurately determining whether two objects are colliding. After the broad phase quickly eliminates pairs of objects that are unlikely to collide, the narrow phase examines the remaining pairs for potential collisions. This can often be more computationally expensive.

GJK (Gilbert-Johnson-Keerthi): The GJK algorithm calculates the distance between convex shapes and detects collisions when this distance is zero. It operates by incrementally constructing a "Minkowski difference" between two objects to efficiently determine if they intersect. GJK is widely used for real-time applications due to its accuracy and performance with convex shapes.

MPR (Minkowski Portal Refinement): MPR is another algorithm for detecting collisions between convex shapes, using the Minkowski difference to locate intersections. It differs from GJK in its approach to iteratively refining a portal that represents the collision region. MPR is known for its stability and efficiency, making it a viable alternative to GJK.

Saturating Axis Theorem: SAT determines if two convex shapes are colliding by testing whether a line (axis) exists on which the objects are separated. If such an axis is found, the objects are not colliding; if not, they are. SAT is efficient for convex polygons and polyhedra, making it commonly used in 2D and 3D collision detection.

In our application, we have two types of objects. For the fluid particles in our scene, narrow-phase collision detection is simplified to a distance check. By comparing the center-to-center distance of each pair of particles with the sum of their radii, we can quickly determine if they are colliding. This approach is efficient and well-suited to spherical shapes, as it avoids the need for complex calculations.

For the larger, more complex objects in our scene, we can approximate their shapes by combining multiple oriented bounding boxes (OBBs). This technique simplifies collision detection by enclosing complex geometries within OBBs that can be efficiently tested for collisions. The Separating Axis Theorem (SAT) is the most efficient narrow-phase collision detection algorithm for handling these oriented bounding boxes. SAT is particularly effective for OBBs because it can handle their orientation and provides accurate collision detection without excessive computational cost, making it well-suited for our simulation's larger and more complex objects.

COLLIDER TYPES

In order to have collision detection, you must first build colliders. Colliders are bounding shapes that can be used to represent the surface of an object for the purposes for collision detection. They can vary from basic shapes like spheres and boxes to complex meshes. We will start by implementing the following basic shapes, eventually expanding into more complex ones. It is worth noting that these shapes can be combined to create colliders for different objects and that there are equivalent analogs for 2 and 3 dimensions. There are also numerous methods for implementation.

Bounding Box

- Position (x, y, z) – Coordinates of the box's center
- Box Size (L, W, H) – Dimensions of the box

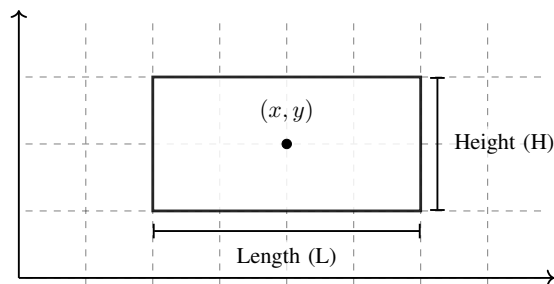


FIGURE 5. Box collider

Bounding Sphere

- Center Point (x, y, z) – Coordinates of the sphere
- Radius (r) – Size of sphere

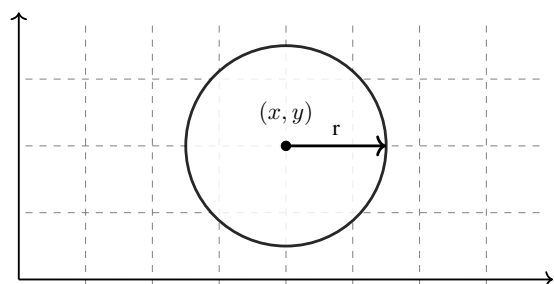


FIGURE 6. Bounding sphere collider

Smoothed Particle

(Used in fluid simulations)

- Position (x, y, z) – Coordinates of particle center
- Distance (r) – Distance from center
- Smoothing Length (h) – Area of influence
- Smoothing Function – Strength of repulsive force

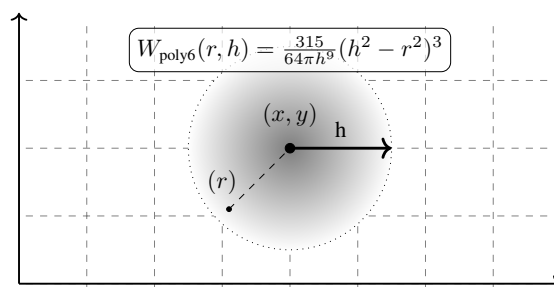


FIGURE 7. Smooth sphere collider with Poly6 kernel

Oriented Bounding Box

- Position (x, y, z) – Coordinates of the box's center
- Box Size (L, W, H) – Dimensions of the box
- Rotation (θ) – Rotation of the box

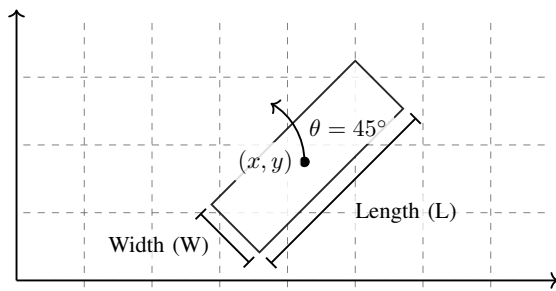


FIGURE 8. Oriented Box Collider

GPU ARCHITECTURE

CPUs vs. GPUs

CPUs excel at performing sequential calculations. GPUs excel at performing parallel calculations through a unique architecture called SIMT (single instruction, multiple threads). This section will define concepts and build towards an understanding of SIMT.

Sequential execution A CPU performing instructions in sequence is sequential execution.

Multicore CPUs were later divided into multiple cores, with each core performing sequential execution of a different set of instructions—the set of instructions being called a “thread”. This is depicted in Figure 9.[2]

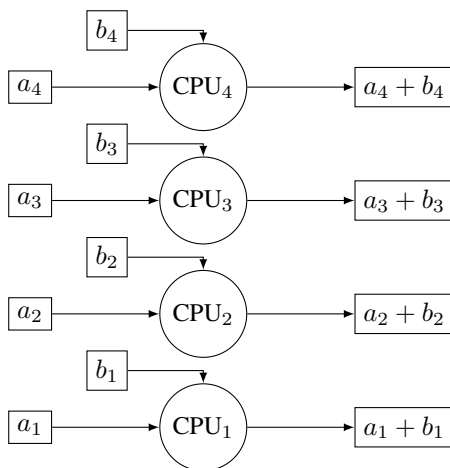


FIGURE 9. Depiction of multicore addition

SIMD Single instruction multiple data (SIMD) means a single instruction performing several operations at once. For example, a SIMD addition instruction is depicted in Figure 10. In this way, one core can perform multiple operations.[2]

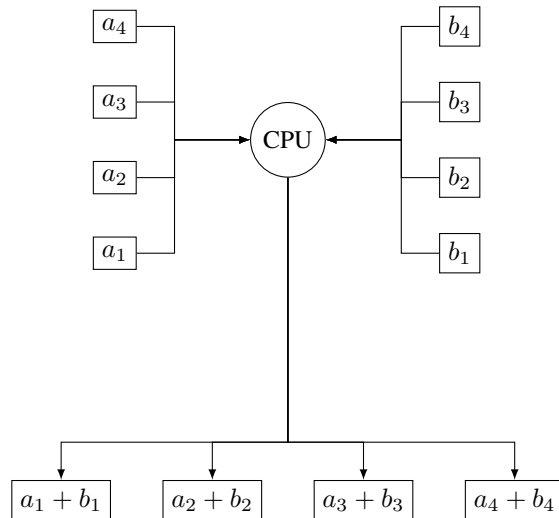


FIGURE 10. Depiction of SIMD addition

SIMT Single Instruction, Multiple Threads (SIMT) combines SIMD with multicore processing. A task is divided into threads, which are grouped by the GPU into sets of 32 or 64 threads, known as a *warp* (or *wavefront* or *cooperative thread array*). Each warp is assigned to a GPU core, where SIMD instructions are used for parallel processing within the warp [2].

Rather than assigning each thread a separate program counter, then branching to indicate conditional execution, a GPU will disable individual threads from executing (this is referred to as *masking* a thread). A masked thread will still be processed by the GPU, but will not have an impact in summary, on the processor state.[2] This is similar to an if-then-else instruction on the ARM7 series of processors.

Optimization concerns

Wave coherency Because GPUs use SIMT, they are fast when all threads in a wave are executing the same instruction—this quality is called *wave coherency*. For many practical problems, threads end up diverging in state, causing the wave to lose coherency. A diverging thread will cause all other threads in a wave to waste time executing useless code.

Data storage

Device memory: The simplest form of data storage on a GPU is the DRAM located on the card. This behaves similarly to normal computer RAM, but is optimized for bandwidth so that the CPU-GPU can both read/write from that memory.

Registers: Both GPUs and CPUs have registers. However, GPUs have separate register spaces for waves (*SGPRs*) and for individual threads in a wave (*VGPRs*). VGPRs are obtained by threads from a limited allocation pool; the pool does not have enough VGPRs for every thread in a wavefront to use the maximum amount of registers it theoretically has access to. This means that *register pressure*—the state where VGPRs bottleneck wave dispatch—can affect performance in a way that doesn't matter for CPUs.

Shared Data: GPUs have a memory space called *shared memory* or a *local data share*. This memory is only shared between waves physically located nearby on the GPU, making a shared memory access lower latency than a device memory access.

For many algorithms, it is possible to arrange threads such that inter-thread communication can be done in shared memory instead of device memory. Additionally, writing to shared memory can alleviate register pressure.

HETEROGENEOUS CPU-GPU INTEGRATION

General Approaches

General implementations of heterogeneous CPU-GPU approaches to various problems share a few common traits [3], [4], [5], [6]:

- 1) Highly parallelizable tasks are kept almost exclusively on the GPU (e.g. high-dimension vector computations, large matrix operations, LTI system processing etc.)
- 2) Sparse and/or irregular parallelizable tasks may be moved to the CPU
- 3) Sequential tasks may vary, and are usually:
 - a) Written into parallel versions of these tasks for GPU execution if they are large enough and doing so is possible (e.g. summations, sorting algorithms – sequential problems with parallel solutions)
 - b) Left on the CPU if they are short or if parallelizing the task is hard/impossible

(i.e. trivial sequential code before dispatching, unpredictable structures such as graph traversal)

in summary, Of primary interest to us are (2) and (3b) above, and the high-level justification for when each is applicable can be simplified as such:

- For (2), the opportunity gain in performance from enabling the GPU to perform other calculations instead must be greater than the cost in latency that is incurred
 - Beating latency may not be difficult if the task is adequately sparse, however gains in performance may not be meaningful unless such tasks comprise a majority of the GPU time actually in use
- For (3b), such tasks must either be large enough in quantity or be impossible to parallelize (have unpredictable structures with true data dependencies) or else any improvements would be negligible or meaningless when moving it to the CPU
 - If a parallelizable problem is left sequential, of course it will run faster if the CPU is used
 - If the sequential problem is too short, performance improvements may be insignificant

Both [3] and [4] utilize strategies aligning with (2), with [4] specifically tackling an implementation of the finite-difference time domain (FDTD) method for modelling electrodynamics, where a sub-gridding method was used to model electric and magnetic vector fields in a 2D or 3D space. Forces in a given cell could be calculated by summing the individual contribution of vector forces in adjacent cells, however researchers in the paper found that sparse grid sections (with cells containing fewer vectors to sum) could be dispatched to the CPU instead of the GPU and saw a 7.45% increase in simulation speed. In regions of dense-sparse cell boundaries (requiring CPU-GPU communication), they instead opted for linear interpolation to approximate cross-boundary contributions, presumably because such calculations would have been computationally expensive due to latency overheads.

SKMD and Problem-Specific Approaches

The general approaches outlined above however, have been criticized for being slow and expensive on their own, namely due to the additional data transfer overheads and poor generalization beyond single CPU-GPU models [6] [7]. The concept of SKMD (Single-

Kernel Multiple Devices) in particular can alleviate both of the aforementioned issues by optimizing CPU-GPU workloads through a workload performance predictor and partitioning and dispatching kernels to mixed device setups (e.g. iGPU, multiple CPU), and claims average performance improvements of 28% on a single CPU, 2 asymmetric GPU setup when compared to a fastest device setup [7].

While SKMD proposes a general solution, papers such as [6] go in the opposite direction, presenting precise solutions leveraging particular hardware specifications and algorithmic optimizations to achieve a tenfold reduction in convergence time for solving of time-evolution partial differential equations.

A Particle Collision Approach - Density-Based Grid Division

We propose a simple heterogeneous collision detection model for particle collisions that leverages our model's fluid density values to help determine CPU-GPU workload split. At a high level, our collision detection algorithm for fluids is a two-phase algorithm, consisting of a broad-phase spatial subdivision method followed by a narrow-phase distance comparison (as our collisions are between circular particles). To utilize CPU resources, we plan to dispatch a subset of the narrow-phase collision detection (alongside associated fluid vector calculations) to the CPU with a simple workload predictor approach that generates a dynamic threshold value for fluid density (across each hashed cell in our fluid grid) to determine whether a given cell should be dispatched to the CPU or GPU on the next frame. A simple diagram of our approach, at a high level, is given in Figure 11.

Important details to note are that our broad-phase algorithm has several areas of flexibility, notable in the sorting algorithm, hashing method, and whether we implement quad or octrees for additional precision; and that narrow-phase collisions implicate additional vector calculations proportional to the number of particles that are colliding simultaneously for a given grid cell.

Additionally, the basis for calculating our workload threshold value is still tentative, and obtaining it dynamically may involve either SKMD directly, or otherwise some SKMD-esque solution (depending on tooling capabilities). Planned approaches for implementation may involve compiling of C++ .dll's for interfacing with C# code. As a result, how exactly we plan on merging the data (and on which device) is also tentative.

SMOOTH PARTICLE HYDRODYNAMICS

We will be implementing (SPH) which is a particle-based simulation technique for fluid dynamics, approximating fluid properties through particle interactions. Based on the Navier-Stokes equations, SPH reformulates these to use smoothed approximations over neighboring particles, calculating forces like pressure and viscosity. Density is determined by summing the mass contributions of neighboring particles, weighted by a kernel function, ensuring local fluid density is smoothly interpolated. SPH estimates pressure forces from particle positions and densities, maintaining stable fluid behavior [8].

FURTHER PERFORMANCE OPTIMIZATIONS

The following techniques focus on enhancing optimizing collision detection across the CPU & GPU via parallel efficiency, reducing data transfer bottlenecks, and maintaining high wave coherence on the GPU.

Combining broad-phase methods such as spatial hashing with dynamic data structures like BVHs on the GPU can enhance collision detection for dynamic scenes. In this approach, spatial hashing efficiently manages uniform particles, while BVH handles complex or rotating objects.

To optimize wave coherence, collision checks can be grouped by similar object types (e.g., particle-particle, object-particle)

Memory access patterns significantly impact collision detection performance through both temporal and spatial locality. (Principle of Locality)

Organizing particle data in Structure-of-Arrays format (separate arrays for positions, velocities, and other attributes) rather than **AoS** (class/object instances) enables coalesced memory access patterns and better SIMD utilization.

Aligning data structures to cache line boundaries (typically 64 bytes) and padding arrays to avoid false sharing between threads reduces cache coherency overhead in parallel execution.

Implementing space-filling curves (e.g., Z-order, Hilbert) for spatial partitioning ensures that spatially proximate objects are also proximate in memory, improving prefetch efficiency and cache hit rates.

Coordinating GPU thread blocks to process spatially coherent regions maximizes L1 cache utilization and reduces global memory bandwidth requirements.

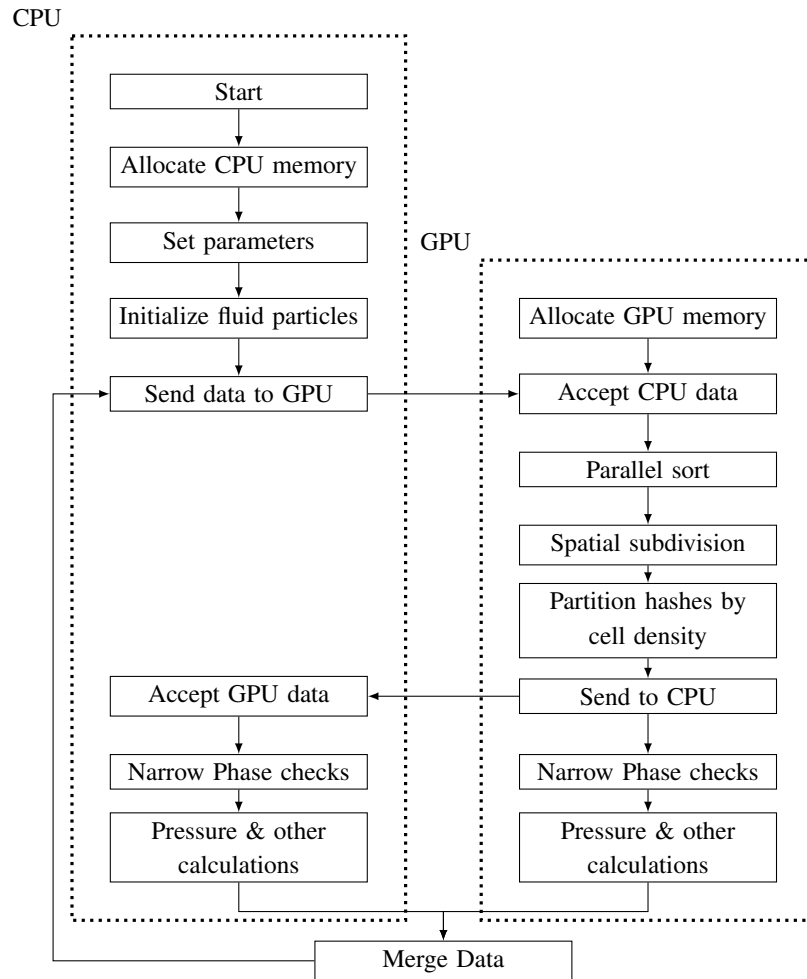


FIGURE 11. High-level Density Approach Flowchart

COLLISION RESOLUTION ALGORITHMS

It is one thing to detect that a collision has occurred, the other side of the problem is how to resolve collisions in a satisfying way. For example, if a bowling ball was thrown at a bowling pin, one might expect the pins to go flying, taking into account their inertia, gravity, momentum, mass, and other physical constraints. In order to accomplish this, it is not enough to detect that a collision has occurred, it is also imperative to take into account the "magnitude" of the collision. This is where signed distance fields come into play.

Because all calculations in an interactive simulation happen in relation to a frame or a fixed time step, in adjacent frames, two colliders may suddenly overlap. When this happens, in order to 'repel' the colliders away from each other, one needs to calculate

the trajectory that each collider should move to stop overlapping each other. This, as well as signed distance fields will be a topic of our next paper.

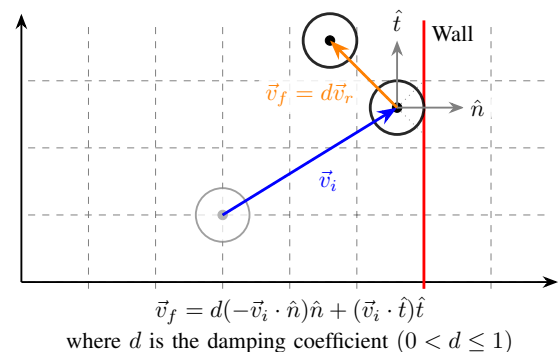


FIGURE 12. Circle collider 45-degree reflection with damping coefficient

PRELIMINARY BENCHMARKS

Frame time is used instead of FPS because FPS is a frequency measure, which is less suited for comparing computational times. For example, a drop from 300 to 260 FPS might seem significant (a 40 FPS difference), but the actual time difference per frame is only 0.51 ms ($1000/300 - 1000/260$). This is roughly equivalent to a change from 60 to 62 FPS. This shows how FPS can misrepresent timing differences at higher frequencies.

Our benchmark system targets a soft-real-time threshold of 60 FPS (16.67 ms per frame), aiming for consistent frame times to avoid visual stutters.

However, frame time has limitations, it doesn't capture variance or frame-time stutter, which can cause perceptible jitters and disrupt smoothness. Thus, for comprehensive analysis, metrics that capture variance, will be included in future papers.

Configuration / Specifications	CPU	GPU
Laptop	i7-7700HQ	GTX 1060
Phone	Snapdragon 888	Adreno 660

TABLE 1. Configuration Specifications

Configuration / Particle Count	250 Particles	500 Particles	1000 Particles
Laptop CPU only	20.76ms	35.86ms	842ms
Laptop iGPU	6.52ms	9.67ms	11.07ms
Laptop GPU	0.69ms	0.75ms	1.02ms
Phone CPU	9.90ms	94.85ms	1100ms
Phone GPU	3.84ms	4.75ms	5.42ms

TABLE 2. Benchmark Results of Total Frametime for Different Platforms and Particle Counts

Theoretical yield is calculated with a linear interpolation assuming that the CPU becomes fully occupied for the time that it is idle in the GPU only configurations. Thus as we optimize the GPU, the max yield should decrease.

Configuration	Theoretical Max Yield Percentage
Laptop CPU + GPU	$\approx 1.23\%$
Laptop CPU + iGPU	$\approx 13.33\%$
Phone CPU + GPU	$\approx 13.68\%$

TABLE 3. Theoretical Max Performance Benefit from Heterogeneous Implementation

IN CONCLUSION,

Our research highlights significant potential of CPU-GPU heterogeneous integration for collision detection in particle simulations. Preliminary benchmarks indicate up to 13% improvement in performance on low-power devices like laptops and smartphones.

REFERENCES

1. Y. R. Serpa and M. A. F. Rodrigues, "Broadmark: A testing framework for broad-phase collision detection algorithms," *Computer graphics forum*, vol. 39, no. 1, pp. 436–449, 2020.
2. T. M. Aamodt, W. W. Lun Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*, ser. Synthesis Lectures on Computer Architecture. Cham: Springer International Publishing.
3. M. Tayyub and G. N. Khan, "Heterogeneous design and efficient CPU-GPU implementation of collision detection," *IADIS International Journal on Computer Science and Information Systems*, vol. 14, no. 2, pp. 25–40, 2019.
4. C. Liu, J. Feng, and M. Fang, "Heterogeneous cpu-gpu accelerated parallel subgridding fdtd algorithm," in *2022 International Applied Computational Electromagnetics Society Symposium (ACES-China)*, 2022, pp. 1–3.
5. A. Gondhalekar, P. Sathre, and W. chun Feng, "Hybrid cpu-gpu implementation of edge-connected jaccard similarity in graph datasets," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC23)*, Denver, CO, USA, November 2023. [Online]. Available: https://sc23.supercomputing.org/proceedings/tech_poster/tech_poster_pages/rpost221.html
6. T. Ichimura, K. Fujita, M. Hori, L. Madgededara, J. Wells, A. Gray, I. Karlin, and J. Linford, "Heterogeneous computing in a strongly-connected cpu-gpu environment: fast multiple time-evolution equation-based modeling accelerated using data-driven approach," 2024. [Online]. Available: <https://arxiv.org/abs/2409.20380>
7. J. Lee, M. Samadi, Y. Park, and S. Mahlke, "Skmd: Single kernel on multiple devices for transparent cpu-gpu collaboration," *ACM Transactions on Computer Systems*, vol. 33, 08 2015.
8. D. Koschier, J. Bender, B. Solenthaler, and M. Teschner. (2019) Smoothed particle hydrodynamics techniques for the physics based simulation of fluids and solids. [Online]. Available: https://sph-tutorial.physics-simulation.org/pdf/SPH_Tutorial.pdf